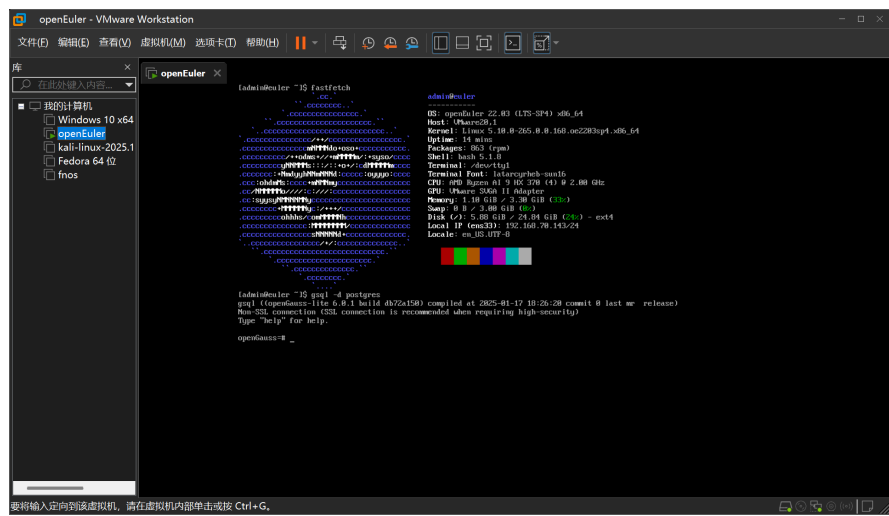


# 数据库课内实验报告

姓名：吴浩哲      班级：网安2201      学号：2223612444

## 一、openGauss的安装

使用本地虚拟机平台VMWare Workstation安装openEuler，在openEuler中安装openGauss



为方便操作，后续实验使用vscode通过ssh连接到虚拟机，编写好SQL命令后复制粘贴到集成终端运行

## 二、数据库的创建和基础信息录入

### 2.1 创建并录入表S444

```
mydb=# CREATE TABLE S444(
mydb(#      S# VARCHAR(8) PRIMARY KEY,
mydb(#      SNAME VARCHAR(40),
mydb(#      SEX VARCHAR(3),
mydb(#      BDATE DATE,
mydb(#      HEIGHT FLOAT4,
mydb(#      DORM VARCHAR(40)
mydb(# );
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index "s444_pkey" for table "s444"
CREATE TABLE
mydb=# INSERT INTO S444 (S#, SNAME, SEX, BDATE, HEIGHT, DORM)
mydb=# VALUES ('01032010', '王涛', '男', '2003-4-5', 1.72, '东 6 舍 221'),
mydb=# ('01032023', '孙文', '男', '2004-6-10', 1.80, '东 6 舍 221'),
mydb=# ('01032001', '张晓梅', '女', '2003-11-17', 1.58, '东 1 舍 312'),
mydb=# ('01032005', '刘静', '女', '2003-1-10', 1.63, '东 1 舍 312'),
mydb=# ('01032112', '董喆', '男', '2003-2-20', 1.71, '东 2 舍 221'),
mydb=# ('03031011', '王倩', '女', '2004-12-20', 1.66, '东 2 舍 104'),
mydb=# ('03031014', '赵思扬', '男', '2002-6-6', 1.85, '东 18 舍 421'),
mydb=# ('03031051', '周剑', '男', '2002-5-8', 1.68, '东 18 舍 422'),
mydb=# ('03031009', '田菲', '女', '2003-8-11', 1.60, '东 2 舍 104'),
mydb=# ('03031033', '蔡明明', '男', '2003-3-12', 1.75, '东 18 舍 423'),
mydb=# ('03031056', '曹子衿', '女', '2005-12-15', 1.65, '东 2 舍 305');
INSERT 0 11
```

### 2.2 创建并录入表C444

```

mydb=# CREATE TABLE C444(
mydb(#   C# VARCHAR(5) PRIMARY KEY,
mydb(#   CNAME VARCHAR(60),
mydb(#   PERIOD INT2,
mydb(#   CREDIT FLOAT4,
mydb(#   TEACHER VARCHAR(40)
mydb(# );
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index "c444_pkey" for table "c444"
CREATE TABLE
mydb=# INSERT INTO C444 (C#, CNAME, PERIOD, CREDIT, TEACHER)
mydb=# VALUES ('CS-01', '数据结构', 60, 3, '张军'),
mydb=#          ('CS-02', '计算机组成原理', 80, 4, '王亚伟'),
mydb=#          ('CS-04', '人工智能', 40, 2, '李蕾'),
mydb=#          ('CS-05', '深度学习', 40, 2, '崔昀'),
mydb=#          ('EE-01', '信号与系统', 60, 3, '张明'),
mydb=#          ('EE-02', '数字逻辑电路', 100, 5, '胡海东'),
mydb=#          ('EE-03', '光电子学与光子学', 40, 2, '石韬');
INSERT 0 7

```

## 2.3 创建并录入表SC444

```

mydb=# CREATE TABLE SC444(
mydb(#   S# VARCHAR(8),
mydb(#   C# VARCHAR(5),
mydb(#   GRADE FLOAT4,
mydb(#   PRIMARY KEY (S#, C#),
mydb(#   FOREIGN KEY (S#) REFERENCES S444(S#),
mydb(#   FOREIGN KEY (C#) REFERENCES C444(C#)
mydb(# );
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index "sc444_pkey" for table "sc444"
CREATE TABLE
mydb=# INSERT INTO SC444 (S#, C#, GRADE)
mydb=# VALUES ('01032010', 'CS-01', 82.0),
mydb=#          ('01032010', 'CS-02', 91.0),
mydb=#          ('01032010', 'CS-04', 83.5),
mydb=#          ('01032001', 'CS-01', 77.5),
mydb=#          ('01032001', 'CS-02', 85.0),
mydb=#          ('01032001', 'CS-04', 83.0),
mydb=#          ('01032005', 'CS-01', 62.0),
mydb=#          ('01032005', 'CS-02', 77.0),
mydb=#          ('01032005', 'CS-04', 82.0),
mydb=#          ('01032023', 'CS-01', 55.0),
mydb=#          ('01032023', 'CS-02', 81.0),
mydb=#          ('01032023', 'CS-04', 76.0),
mydb=#          ('01032112', 'CS-01', 88.0),
mydb=#          ('01032112', 'CS-02', 91.5),
mydb=#          ('01032112', 'CS-04', 86.0),
mydb=#          ('01032112', 'CS-05', NULL),
mydb=#          ('03031033', 'EE-01', 93.0),
mydb=#          ('03031033', 'EE-02', 89.0),
mydb=#          ('03031009', 'EE-01', 88.0),
mydb=#          ('03031009', 'EE-02', 78.5),
mydb=#          ('03031011', 'EE-01', 91.0),
mydb=#          ('03031011', 'EE-02', 86.0),
mydb=#          ('03031051', 'EE-01', 78.0),
mydb=#          ('03031051', 'EE-02', 58.0),
mydb=#          ('03031014', 'EE-01', 79.0),
mydb=#          ('03031014', 'EE-02', 71.0);
INSERT 0 26

```

## 2.4 查询各表及属性信息

```

mydb=# SELECT relname AS 表名, relkind AS 类型, relpersistence AS 存储方式
mydb=# FROM pg_class
mydb=# WHERE relname='s444'
mydb=#        OR relname='c444'
mydb=#        OR relname='sc444';
 表名 | 类型 | 存储方式
-----+-----+-----
s444  | r    | p
c444  | r    | p
sc444 | r    | p
(3 rows)

```

## 三、增删改查

### 3.1 查

#### 3.1.1 查询电子工程系（EE）所开课程的课程编号、课程名称及学分数

```
mydb=# SELECT C#, CNAME, CREDIT
mydb=# FROM C444
mydb=# WHERE C# LIKE 'EE%';
```

| c#    | cname   | credit |
|-------|---------|--------|
| EE-01 | 信号与系统   | 3      |
| EE-02 | 数字逻辑电路  | 5      |
| EE-03 | 光子学与光子学 | 2      |

(3 rows)

#### 3.1.2 查询未选课程“CS-02”的女生学号及其已选各课程编号、成绩

```
mydb=# SELECT S444.S#, C#, GRADE
mydb=# FROM S444 LEFT JOIN SC444
mydb=# ON S444.S#=SC444.S#
mydb=# WHERE SEX='女' AND S444.S# NOT IN (
mydb=# SELECT S#
mydb=# FROM SC444
mydb=# WHERE C#='CS-02'
mydb=# );
```

| s#       | c#    | grade |
|----------|-------|-------|
| 03031011 | EE-01 | 91    |
| 03031011 | EE-02 | 86    |
| 03031009 | EE-01 | 88    |
| 03031009 | EE-02 | 78.5  |
| 03031056 |       |       |

(5 rows)

#### 3.1.3 查询 2004 年 ~ 2005 年出生学生的基本信息

```
mydb=# SELECT *
mydb=# FROM S444
mydb=# WHERE EXTRACT(YEAR FROM BDATE) BETWEEN 2004 AND 2005;
```

| s#       | sname | sex | bdate               | height | dorm      |
|----------|-------|-----|---------------------|--------|-----------|
| 01032023 | 孙文    | 男   | 2004-06-10 00:00:00 | 1.8    | 东 6 舍 221 |
| 03031011 | 王倩    | 女   | 2004-12-20 00:00:00 | 1.66   | 东 2 舍 104 |
| 03031056 | 曹子衿   | 女   | 2005-12-15 00:00:00 | 1.65   | 东 2 舍 305 |

(3 rows)

#### 3.1.4 查询每位学生的学号、学生姓名及其已选课程的学分总数

```
mydb=# SELECT S444.S#, SNAME, SUM(CREDIT) AS TOTAL_CREDIT
mydb=# FROM S444
mydb=# LEFT JOIN SC444 ON S444.S#=SC444.S#
mydb=# LEFT JOIN C444 ON C444.C#=SC444.C#
mydb=# GROUP BY S444.S#;
```

| s#       | sname | total_credit |
|----------|-------|--------------|
| 03031014 | 赵思扬   | 8            |
| 03031033 | 蔡明明   | 8            |
| 01032023 | 孙文    | 9            |
| 03031009 | 田菲    | 8            |
| 03031056 | 曹子衿   |              |
| 03031011 | 王倩    | 8            |
| 01032112 | 董喆    | 11           |
| 03031051 | 周剑    | 8            |
| 01032001 | 张晓梅   | 9            |
| 01032010 | 王涛    | 9            |
| 01032005 | 刘静    | 9            |

(11 rows)

### 3.1.5 查询选修课程“CS-01”的学生中成绩第二高的学生学号

```
mydb=# SELECT S#
mydb=# FROM (
mydb=#     SELECT S#, GRADE, DENSE_RANK() OVER (ORDER BY GRADE DESC) AS grade_rank
mydb=#     FROM SC444
mydb=#     WHERE C# = 'CS-01'
mydb=# ) AS ranked
mydb=# WHERE grade_rank = 2;
s#
-----
01032010
(1 row)
```

### 3.1.6 查询平均成绩超过“王涛”同学的学生学号、姓名和平均成绩，并按学号进行降序排列

```
mydb=# SELECT S444.S#, SNAME, AVG(GRADE) AS AVG_GRADE
mydb=# FROM S444 JOIN SC444 ON S444.S#=SC444.S#
mydb=# GROUP BY S444.S#
mydb=# HAVING AVG_GRADE>(
mydb=#     SELECT AVG(GRADE)
mydb=#     FROM S444 JOIN SC444 ON S444.S#=SC444.S#
mydb=#     WHERE SNAME='王涛'
mydb=# )
mydb=# ORDER BY S444.S# DESC;
s# | sname | avg_grade
-----+-----+-----
03031033 | 蔡明明 | 91
03031011 | 王倩 | 88.5
01032112 | 董喆 | 88.5
(3 rows)
```

### 3.1.7 查询选修了计算机专业全部课程（课程编号为“CS-xx”）的学生姓名及已获得的学分总数

```
mydb=# SELECT SNAME, SUM(CREDIT) AS TOTAL_CREDIT
mydb=# FROM S444
mydb=# JOIN SC444 ON S444.S#=SC444.S#
mydb=# JOIN C444 ON C444.C#=SC444.C#
mydb=# WHERE GRADE>=60 AND S444.S# IN (
mydb=#     SELECT S444.S#
mydb=#     FROM S444
mydb=#     WHERE NOT EXISTS (
mydb=#         -- 外层查询筛选出所有计算机课程
mydb=#         SELECT C444.C#
mydb=#         FROM C444
mydb=#         WHERE C444.C# LIKE 'CS-%'
mydb=#         AND NOT EXISTS (
mydb=#             -- 内层 NOT EXISTS 确保学生没有未选修的计算机课程
mydb=#             SELECT *
mydb=#             FROM SC444
mydb=#             WHERE SC444.S# = S444.S#
mydb=#             AND SC444.C# = C444.C#
mydb=#         )
mydb=#     )
mydb=# )
mydb=# GROUP BY S444.S#;
sname | total_credit
-----+-----
董喆 | 9
(1 row)
```

### 3.1.8 查询选修了3门以上课程（包括3门）的学生中平均成绩最高的同学学号及姓名

```

mydb=# FROM S444
mydb=# WHERE S# IN (
mydb=#     SELECT S#
mydb=#     FROM (
mydb=#         SELECT S#, AVG(GRADE) AS AVG_GRADE,
mydb=#         DENSE_RANK() OVER (ORDER BY AVG_GRADE DESC) AS grade_rank
mydb=#         FROM SC444
mydb=#         GROUP BY S#
mydb=#         HAVING COUNT(*)>=3
mydb=#     ) AS ranked
mydb=#     WHERE grade_rank = 1
mydb=# );
 s# | sname
-----+-----
01032112 | 董喆
(1 row)

```

## 3.2 增

分别在S×××和C×××表中加入记录('01032005', '刘竞', '男', '2003-12-10', 1.75, '东 14 舍 312')及('CS-03', "离散数学", 64, 4, '陈建明')

```

mydb=# INSERT INTO S444 (S#, SNAME, SEX, BDATE, HEIGHT, DORM)
mydb=# VALUES ('01032005', '刘竞', '男', '2003-12-10', 1.75, '东 14 舍 312');
ERROR:  duplicate key value violates unique constraint "s444_pkey"
DETAIL:  Key ("s#")=(01032005) already exists.

```

```

mydb=# INSERT INTO C444 (C#, CNAME, PERIOD, CREDIT, TEACHER)
mydb=# VALUES ('CS-03', '离散数学', 64, 4, '陈建明');
INSERT 0 1

```

## 3.3 删

将S444表中已修学分数大于60的学生记录删除

```

mydb=# DELETE FROM S444
mydb=# WHERE S# IN (
mydb=#     SELECT S#
mydb=#     FROM C444, SC444
mydb=#     WHERE C444.C#=SC444.C#
mydb=#     GROUP BY S#
mydb=#     HAVING SUM(CREDIT)>60
mydb=# );
DELETE 0

```

## 3.4 改

将“张明”老师负责的“数字电子技术”课程的学时数调整为36，同时增加一个学分

```

mydb=# UPDATE C444
mydb=# SET PERIOD=36, CREDIT=CREDIT+1
mydb=# WHERE TEACHER='张明' AND CNAME='数字电子技术';
UPDATE 0

```

## 3.5 视图

3.5.1 居住在“东18舍”的男生视图，包括学号、姓名、出生日期、身高等属性

```

mydb=# CREATE VIEW view1 AS
mydb=# SELECT S#, SNAME, BDATE, HEIGHT
mydb=# FROM S444
mydb=# WHERE SEX='男' AND DORM LIKE '东 18 舍%';
CREATE VIEW
mydb=# SELECT * FROM view1;
s#      | sname      | bdate      | height
-----+-----+-----+-----
03031014 | 赵思扬     | 2002-06-06 00:00:00 | 1.85
03031051 | 周剑       | 2002-05-08 00:00:00 | 1.68
03031033 | 蔡明明     | 2003-03-12 00:00:00 | 1.75
(3 rows)

```

3.5.2 张明”老师所开设课程情况的视图，包括课程编号、课程名称、平均成绩等属性

```

mydb=# CREATE VIEW view2 AS
mydb=# SELECT C444.C#, CNAME, AVG(GRADE) AS avg_grade
mydb=# FROM C444, SC444
mydb=# WHERE C444.C#=SC444.C# AND TEACHER='张明'
mydb=# GROUP BY C444.C#;
CREATE VIEW
mydb=# SELECT * FROM view2;
c#      | cname      | avg_grade
-----+-----+-----
EE-01   | 信号与系统 | 85.8
(1 row)

```

3.5.3 所有选修了“人工智能”课程的学生视图，包括学号、姓名、成绩等属性

```

mydb=# CREATE VIEW view3 AS
mydb=# SELECT S444.S#, SNAME, GRADE
mydb=# FROM S444, C444, SC444
mydb=# WHERE S444.S#=SC444.S# AND C444.C#=SC444.C# AND CNAME='人工智能';
CREATE VIEW
mydb=# SELECT * FROM view3;
s#      | sname      | grade
-----+-----+-----
01032010 | 王涛       | 83.5
01032001 | 张晓梅     | 83
01032005 | 刘静       | 82
01032023 | 孙文       | 76
01032112 | 董喆       | 86
(5 rows)

```

## 四、外部程序接口

### 4.1 数据生成

#### 4.1.1 S表

利用字典的键唯一性生成唯一学号

```

# 通过字典的键唯一性，快速生成唯一学号
unique_combinations = {}
while len(unique_combinations) < 5000:
    s = rd.randint(10000000,50000000)
    unique_combinations[s] = True # 键自动去重

```

使用库函数生成符合要求的姓名、性别、生日，其中身高满足正态分布

```

import mingzi as mz
from faker import Faker

# 生成名字和性别

```

```
temp = mz.mingzi(female_rate=0.49, show_gender=True, single_rate=0.3)

# 依据性别生成身高
if (temp[0][1] == "女"):
    row.append(round(rd.gauss(162, 6) / 100, 2))
else:
    row.append(round(rd.gauss(175, 7) / 100, 2))

# 生成生日
row.append(str(fk.date_of_birth(minimum_age=18, maximum_age=22)))
```

### 随机生成宿舍

```
# 生成宿舍
dorm = rd.choice(["东 ", "西 "]) + str(rd.randint(1, 24)) + " 舍 "
dorm += str(rd.randint(1, 8) * 100 + rd.randint(1, 30))
```

### 将生成的数据存入csv文件

```
with open('stu.csv', 'w', newline='') as file:
    stu = csv.writer(file)
    stu.writerows(rows)
```

## 4.1.2 C表

### 从excel表中读取并处理数据

```
# 读取
code = table.cell_value(r, SH)
# 确定小编号
if (code == '计算机学院'):
    no = "%02d" % (table.cell_value(r, N0) + 5)
elif (code == '信息与电子学院'):
    no = "%02d" % (table.cell_value(r, N0) + 3)
else:
    no = "%02d" % (table.cell_value(r, N0))
# 组合产生课程编号
cno = re.sub("|".join(translate_dict.keys()), translate_func, code) + '-' + no
row.append(cno)
# 读取课程名
row.append(table.cell_value(r, CNAME))
# 读取学时
row.append(table.cell_value(r, PERIOD))
```

```
# 读取学分
row.append(table.cell_value(r, CREIDT))
# 读取上课教师
row.append(table.cell_value(r, TEACHER))
```

将生成的数据存入csv文件

```
with open('course.csv', 'w', newline='') as file:
    course = csv.writer(file)
    course.writerows(rows)
```

### 4.1.3 SC表

从已生成的S表和C表中读取学号和课程号信息

```
# 读取学号信息
with open('stu.csv', 'r') as stu_csv:
    reader = csv.DictReader(stu_csv)
    stu_list = [row['s#'] for row in reader]

# 读取课程号信息
with open('course.csv', 'r') as course_csv:
    reader = csv.DictReader(course_csv)
    course_list = [row['c#'] for row in reader]
```

生成不重复的随机组合

```
# 通过字典的键唯一性，快速生成唯一组合
unique_combinations = {}
while len(unique_combinations) < 200000:
    s = rd.choice(stu_list)
    c = rd.choice(course_list)
    unique_combinations[(s, c)] = True # 键自动去重
```

成绩应满足正态分布，且有上下界，编写对应的生成函数

```
# 有上下界的随机正态分布随机函数
def truncated_gauss(mu = 80, sigma = 12, low = 0, high = 100):
    while True:
        num = rd.gauss(mu, sigma)
```



```
if low <= num <= high:
    return num
```

设定低于50分为无记录

```
for (s, c) in unique_combinations.keys():
    grade = round(truncated_gauss(), 1)
    if grade > 50:
        rows.append([s, c, grade])
    else:
        rows.append([s, c])
```

将生成的数据存入csv文件

```
with open('sc.csv', 'w', newline='') as file:
    sc = csv.writer(file)
    sc.writerows(rows)
```

## 4.1.4 生成数据截图

|      |                                                |     |                                                   |        |                       |
|------|------------------------------------------------|-----|---------------------------------------------------|--------|-----------------------|
| 4980 | 26690095, 萧丹华, 女, 1.6, 2007-04-03, 西 11 舍 703  | 920 | FL-77, 日语写作II, 32, 2, MATSUDA AYA                 | 199985 | 24637872, EM-18, 91.4 |
| 4981 | 42396994, 薛凯泽, 男, 1.72, 2005-03-02, 西 12 舍 804 | 929 | FL-78, 西班牙语文学, 32, 2, LERMA PELAEZ JUAN GONZALO   | 199986 | 25416048, EE-56, 77.6 |
| 4982 | 42789390, 凌笑, 女, 1.54, 2005-12-11, 东 22 舍 327  | 930 | FL-79, 西班牙语报刊阅读, 32, 2, LERMA PELAEZ JUAN GONZALO | 199987 | 33668459, EE-64, 87.4 |
| 4983 | 14589188, 孔彦, 女, 1.58, 2002-10-22, 西 7 舍 324   | 931 | FL-80, 西班牙语听力I, 32, 2, LERMA PELAEZ JUAN GONZALO  | 199988 | 12410014, MA-23, 86.2 |
| 4984 | 17655185, 张婉怡, 女, 1.52, 2006-08-19, 东 8 舍 710  | 932 | FL-81, 应用西班牙语IV, 32, 2, LERMA PELAEZ JUAN GONZALO | 199989 | 42455916, LW-17, 61.0 |
| 4985 | 19436615, 田安, 女, 1.69, 2005-11-30, 西 6 舍 517   | 933 | FL-82, 中国文化概论, 32, 2, LERMA PELAEZ JUAN GONZALO   | 199990 | 49028965, FL-14, 72.7 |
| 4986 | 12755393, 林亦茂, 男, 1.79, 2005-05-03, 西 2 舍 124  | 934 | ME-64, 微系统设计与控制 (全英文), 32, 2, "K1 Young Song, 冯跃" | 199991 | 18006903, AS-27, 93.9 |
| 4987 | 48242041, 殷惠巧, 女, 1.51, 2005-05-09, 西 4 舍 816  | 935 | ME-65, 宏微系统物理学 (全英文), 32, 2, K1 Young Song        | 199992 | 46629817, EE-53, 77.5 |
| 4988 | 34525981, 陈亨, 男, 1.73, 2002-06-06, 西 1 舍 123   | 936 | FL-83, 高级日语II, 32, 2, KAWAMURA NAOKO              | 199993 | 37024356, DA-04, 85.6 |
| 4989 | 18746520, 高晴茹, 女, 1.62, 2004-07-21, 东 10 舍 406 | 937 | FL-84, 日语会话IV, 32, 2, KAWAMURA NAOKO              | 199994 | 25377924, FL-28, 76.5 |
| 4990 | 33059428, 孙芷蕊, 女, 1.62, 2007-02-28, 西 2 舍 125  | 938 | FL-85, 日语写作I, 32, 2, KAWAMURA NAOKO               | 199995 | 39011271, CS-12, 81.5 |
| 4991 | 39674269, 康宛晴, 女, 1.66, 2003-11-18, 东 16 舍 602 | 939 | FL-86, 时事日语, 32, 2, KAWAMURA NAOKO                | 199996 | 39721162, DA-62, 68.9 |
| 4992 | 45671646, 李攀, 男, 1.71, 2003-06-08, 东 13 舍 617  | 940 | FL-87, 德语时事论坛I, 32, 2, JOERG ULRICH               | 199997 | 36516934, LF-14, 75.3 |
| 4993 | 36646914, 张会, 女, 1.61, 2003-06-22, 西 24 舍 230  | 941 | FL-88, 德语学术论文写作, 32, 2, JOERG ULRICH              | 199998 | 24026185, HS-12, 72.8 |
| 4994 | 27943733, 孙翔, 女, 1.7, 2004-09-11, 东 3 舍 223    | 942 | FL-89, 德语听力IV, 32, 2, JOERG ULRICH                | 199999 | 47652793, IE-06, 65.1 |
| 4995 | 20659917, 郑昌, 女, 1.64, 2004-06-20, 西 8 舍 402   | 943 | FL-90, 德语语法分析, 32, 2, JOERG ULRICH                | 200000 | 29071008, MT-37, 89.7 |
| 4996 | 32393587, 孙君后, 男, 1.74, 2003-06-04, 西 9 舍 407  | 944 | FL-91, 西班牙语口语I, 32, 2, Joan Cortada Moguer        | 200001 | 13913607, LF-03, 84.1 |
| 4997 | 14477570, 郭西惠, 女, 1.61, 2006-10-08, 西 11 舍 406 | 945 | FL-92, 西班牙语口语III, 32, 2, Joan Cortada Moguer      | 200002 |                       |
| 4998 | 12414847, 石磊恒, 男, 1.85, 2005-07-20, 东 16 舍 518 | 946 | FL-93, 西班牙语听力IV, 32, 2, Joan Cortada Moguer       |        |                       |
| 4999 | 13704489, 李华, 男, 1.85, 2006-12-21, 西 11 舍 318  | 947 | FL-94, 西班牙语写作II, 32, 2, Joan Cortada Moguer       |        |                       |
| 5000 | 46838759, 刘芷蕊, 女, 1.6, 2006-06-03, 东 11 舍 227  | 948 | FL-95, 西班牙语语法I, 32, 2, Joan Cortada Moguer        |        |                       |
| 5001 | 31688752, 丘婉卿, 女, 1.56, 2002-08-14, 东 1 舍 206  | 949 | EN-68, 智能学原理 (全英文), 32, 2, Issam                  |        |                       |
| 5002 |                                                | 950 | CE-50, 能源化学工程概论, 32, 2, "DAVID ROONEY, 孙旺"        |        |                       |
|      |                                                | 951 | FL-96, 英语口语II, 32, 2, DANIEL ROLFE                |        |                       |
|      |                                                | 952 |                                                   |        |                       |

## 4.2 JDBC连接

### 4.2.1 主程序

存储数据库连接信息、数据表名称、需导入的文件（硬编码关键信息仅限测试环境）

```
public class App {
    private static String JDBC_URL = "jdbc:postgresql:mydb?
useSSL=false&characterEncoding=utf8";
    private static String JDBC_USER = "admin";
    private static String JDBC_PASSWORD = "wuhaozhe@12345";
    private static String S_TABLE = "s444";
```

```

private static String C_TABLE = "c444";
private static String SC_TABLE = "sc444";
private static String S_CSV = "csv/stu.csv";
private static String C_CSV = "csv/course.csv";
private static String SC_CSV = "csv/sc.csv";
}

```

依据不同情景选择创建不同的线程

```

public class App {
    static void insertOnly(String path, String table, int ins_num) {
        String threadName = "insert_" + table;
        Insert in = new Insert(threadName, path, JDBC_URL, JDBC_USER,
JDBC_PASSWORD, table, ins_num);
        in.start();
    }

    static void insertDelete(String path, String table, int ins_num) {
        String threadName1 = "insert_" + table;
        Insert in = new Insert(threadName1, path, JDBC_URL, JDBC_USER,
JDBC_PASSWORD, table, ins_num);
        in.start();
        String threadName2 = "delete_from_" + table;
        Delete de = new Delete(threadName2, JDBC_URL, JDBC_USER, JDBC_PASSWORD,
table, 200, true);
        de.start();
    }

    static void deleteOnly(String table) {
        String threadName = "delete_from_" + table;
        Delete de = new Delete(threadName, JDBC_URL, JDBC_USER, JDBC_PASSWORD,
table, 100, false);
        de.start();
    }
}

```

主函数

```

public class App {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        boolean ifContinue = true;
        while (ifContinue) {
            System.out.println("你要进行的操作:");
            System.out.println("1. 录入S");

```

```

        System.out.println("2. 录入C");
        System.out.println("3. 录入SC(启用多线程删除)");
        System.out.println("4. 录入SC(禁用多线程删除)");
        System.out.println("5. 删除SC");
        System.out.println("0. 退出");
        switch (sc.nextInt()) {
            case 1: insertOnly(S_CSV, S_TABLE, S_NUM); break;
            case 2: insertOnly(C_CSV, C_TABLE, C_NUM); break;
            case 3: insertDelete(SC_CSV, SC_TABLE, SC_NUM); break;
            case 4: insertOnly(SC_CSV, SC_TABLE, SC_NUM); break;
            case 5: deleteOnly(SC_TABLE); break;
            case 0: ifContinue = false; break;
            default: System.out.println("错误");
        }
        Thread.sleep(1000);
    }
    sc.close();
}
}

```

## 4.2.2 Insert线程

采用预编译的方法构造SQL语句，可依据列数动态构造

```

public class Insert implements Runnable {
    // 构造sql语句
    String structureSQL(int headerNum, String[] headerList) {
        String headLine = headerList[0];
        String ques = "?";
        for (int i = 1; i < headerNum; i++) {
            headLine = headLine + "," + headerList[i];
            ques = ques + ",?";
        }
        String sql = "INSERT INTO " + tableName + " (" + headLine + ") VALUES ("
+ ques + ")";
        return sql;
    }
}

```

连接数据库，从csv文件读取数据并完成插入操作

```

public class Insert implements Runnable {
    // 连接数据库
    void connect(String sql, int headerNum, CsvReader csvReader) throws
Exception {

```

```

        int count = 0;
        // 获取连接
        try (Connection conn = DriverManager.getConnection(JDBC_URL, JDBC_USER,
JDBC_PASSWORD)) {
            try (PreparedStatement ps = conn.prepareStatement(sql)) {
                // 从csv文件中读取数据
                while (csvReader.readRecord() && count < INS_NUM) {
                    count++;
                    String[] vs = csvReader.getValues();
                    for (int i = 0; i < headerNum; i++) {
                        String v = (i < vs.length) ? vs[i] : "";
                        // 检查输入数据类型, 完成转换
                        if (v == null || v.trim().isEmpty()) {
                            ps.setNull(i + 1, Types.FLOAT);
                        } else if (v.matches("\\d+(\\.\\d+)?")) {
                            if (v.contains(".")) {
                                ps.setDouble(i + 1, Double.parseDouble(v));
                            } else {
                                ps.setInt(i + 1, Integer.parseInt(v));
                            }
                        } else {
                            ps.setString(i + 1, v);
                        }
                    }
                    ps.executeUpdate();
                }
            }
            // 关闭连接:
            conn.close();
        }
    }
}

```

### 4.2.3 Delete线程

与Insert线程大体相同, 不再赘述

### 4.2.4 插入后截图

在S444表中补充数据至约1000行, 在C444表中补充数据至约100行

|                                                                         |                                                                        |
|-------------------------------------------------------------------------|------------------------------------------------------------------------|
| <pre> mydb=# select count(*) from s444; count ----- 1011 (1 row) </pre> | <pre> mydb=# select count(*) from c444; count ----- 108 (1 row) </pre> |
|-------------------------------------------------------------------------|------------------------------------------------------------------------|

当不启用多线程删除时, 所得SC444

```
mydb=# select count(*) from sc444;
count
-----
20026
(1 row)
```

```
mydb=# select count(*) from sc444
mydb=# where grade<60;
count
-----
910
(1 row)
```

```
mydb=# select count(*) from sc444
mydb=# where grade is null;
count
-----
132
(1 row)
```

恢复后使用相同数据源录入，启用多线程删除，所得SC444

```
mydb=# select count(*) from sc444;
count
-----
19826
(1 row)
```

```
mydb=# select count(*) from sc444
mydb=# where grade<60;
count
-----
733
(1 row)
```

```
mydb=# select count(*) from sc444
mydb=# where grade is null;
count
-----
109
(1 row)
```

可见记录减少200条，全部为成绩低于60分（含NULL）的记录

在S444表中补充数据至约5000行，在C444表中补充数据至约1000行

```
mydb=# select count(*) from s444;
count
-----
5011
(1 row)
```

```
mydb=# select count(*) from c444;
count
-----
958
(1 row)
```

在SC444表中补充数据至约200000行

```
mydb=# select count(*) from sc444;
count
-----
200026
(1 row)
```

```
mydb=# select count(*) from sc444
mydb=# where grade<60;
count
-----
8673
(1 row)
```

```
mydb=# select count(*) from sc444
mydb=# where grade is null;
count
-----
1343
(1 row)
```

## 4.3 更多查询

使用 `explain analyse` 分析语句运行时长

1. 查询未选修课程“CS-02”的女生学号及其已选各课程编号、成绩

```
-- 已有语句
SELECT S444.S#, C#, GRADE
FROM S444 LEFT JOIN SC444
ON S444.S#=SC444.S#
WHERE SEX='女' AND S444.S# NOT IN (
    SELECT S#
    FROM SC444
    WHERE C#='CS-02'
);

-- 修改语句
```

```

SELECT S.S#, SC.C#, SC.GRADE
FROM S444 S
LEFT JOIN SC444 SC ON S.S# = SC.S#
WHERE S.SEX = '女'
AND NOT EXISTS (
    SELECT 1
    FROM SC444
    WHERE S# = S.S# AND C# = 'CS-02'
);

```

NOT EXISTS 在找到第一个匹配项后立即停止扫描，而 NOT IN 需要完整执行子查询，对大表性能更优

经实测，运行时长从81ms降至50ms

## 2. 查询 2004 年 ~ 2005 年出生学生的基本信息

```

-- 已有语句
SELECT *
FROM S444
WHERE EXTRACT(YEAR FROM BDATE) BETWEEN 2004 AND 2005;

--修改语句
SELECT *
FROM S444
WHERE BDATE BETWEEN '2004-01-01' AND '2005-12-31';

```

避免使用 EXTRACT(YEAR FROM BDATE) 函数，让数据库可以直接利用BDATE字段的索引，比使用函数快很多

经实测，运行时长从1.6ms降至1.1ms

## 3. 查询选修了计算机专业全部课程（课程编号为“CS-xx”） 的学生姓名及已获得的学分总数

```

-- 已有语句
SELECT S#
FROM (
    SELECT S#, GRADE, DENSE_RANK() OVER (ORDER BY GRADE DESC) AS grade_rank
    FROM SC444
    WHERE C# = 'CS-01'
) AS ranked

```

```
WHERE grade_rank = 2;
```

```
-- 修改语句
```

```
SELECT S#  
FROM SC444  
WHERE C# = 'CS-01'  
ORDER BY GRADE DESC  
LIMIT 1 OFFSET 1;
```

只需一次排序操作，直接利用排序结果跳过第一条记录，性能最优

经实测，运行时长从29.5ms降至17.5ms

#### 4. 查询选修了3门以上课程（包括3门）的学生中平均成绩最高的同学学号及姓名

```
-- 原有语句
```

```
SELECT S#, SNAME  
FROM S444  
WHERE S# IN (  
    SELECT S#  
    FROM (  
        SELECT S#, AVG(GRADE) AS AVG_GRADE,  
               DENSE_RANK() OVER (ORDER BY AVG_GRADE DESC) AS grade_rank  
        FROM SC444  
        GROUP BY S#  
        HAVING COUNT(*)>=3  
    ) AS ranked  
    WHERE grade_rank = 1  
);
```

```
-- 修改语句
```

```
SELECT S.S#, S.SNAME  
FROM S444 S  
JOIN (  
    SELECT S#, AVG(GRADE) AS AVG_GRADE  
    FROM SC444  
    GROUP BY S#  
    HAVING COUNT(*) >= 3  
    ORDER BY AVG_GRADE DESC
```

```

LIMIT 1
) AS top_student ON S.S# = top_student.S#;

```

只需一次排序操作，通过 LIMIT 1 直接获取最高平均成绩记录，避免使用窗口函数 DENSE\_RANK()，减少排序开销

经实测，运行时长从96ms降至72ms

## 五、备份与恢复

使用gs\_dump命令将数据库备份为一个sql文件

```

[admin@TXAir sql_lab]$ gs_dump mydb -f dump/mydb444.sql
gs_dump[port='5432'][mydb][2025-06-07 00:03:35]: Begin scanning database.
Progress: [=====] 100% (38/37, cur_step/total_step). finish scanning database
gs_dump[port='5432'][mydb][2025-06-07 00:03:35]: Finish scanning database.
gs_dump[port='5432'][mydb][2025-06-07 00:03:35]: Start dumping objects
Progress: [=====] 100% (4029/4029, dumpObjNums/totalObjNums). dump objects
gs_dump[port='5432'][mydb][2025-06-07 00:03:35]: Finish dumping objects
gs_dump[port='5432'][mydb][2025-06-07 00:03:35]: dump database mydb successfully
gs_dump[port='5432'][mydb][2025-06-07 00:03:35]: total time: 252 ms

```

使用 gsql -d yourdb -f dump/mydb391.sql 命令恢复合作同学的数据库，成功恢复后查询表设计

```

-- 查询字段数据类型
SELECT
    a.attname AS "字段名",
    pg_catalog.format_type(a.atttypid, a.atttypmod) AS "数据类型"
FROM
    pg_catalog.pg_attribute a
JOIN
    pg_catalog.pg_class c ON a.attrelid = c.oid
JOIN
    pg_catalog.pg_namespace n ON c.relnamespace = n.oid
WHERE
    c.relname = 'sc391' --表名
    AND a.attnum > 0;

```

| 字段名      | 数据类型                           |
|----------|--------------------------------|
| s#       | character varying(10)          |
| sname    | character varying(40)          |
| sex      | character(3)                   |
| bdate    | timestamp(0) without time zone |
| height   | numeric(3,2)                   |
| dorm     | character varying(40)          |
| (6 rows) |                                |

| 字段名      | 数据类型                  |
|----------|-----------------------|
| c#       | character varying(10) |
| cname    | character varying(70) |
| period   | integer               |
| credit   | numeric(3,1)          |
| teacher  | character varying(40) |
| (5 rows) |                       |

| 字段名      | 数据类型                  |
|----------|-----------------------|
| s#       | character varying(10) |
| c#       | character varying(10) |
| grade    | numeric(4,1)          |
| (3 rows) |                       |

身高、学分等使用定点数，可在保证精度的同时减少资源消耗。学号、课程号使用varchar(10)，为未来可能的位数增加预留空间。课程名长短不一，差别很大，故使用varchar(70)保证足够的长



度，同时尽量减少资源浪费。

## 六、触发器

创建记录表并编写触发器

```
CREATE TABLE BK444 (  
    S# VARCHAR(8),  
    C# VARCHAR(5),  
    GRADE FLOAT4,  
    DDATE DATE  
);  
  
CREATE OR REPLACE FUNCTION TRI_DELETE_FUNC() RETURNS TRIGGER AS  
$$  
DECLARE  
BEGIN  
    INSERT INTO BK444  
    (S#, C#, GRADE, DDATE)  
    VALUES (  
        OLD.S#,  
        OLD.C#,  
        OLD.GRADE,  
        NOW()  
    );  
    RETURN OLD;  
END  
$$ LANGUAGE PLPGSQL;  
  
CREATE TRIGGER delete_trigger  
BEFORE DELETE ON SC444  
FOR EACH ROW  
EXECUTE PROCEDURE tri_delete_func();
```

使用gsql命令导入

```
[admin@TXAir sql_lab]$ gsql -d mydb -f basic/trigger.sql  
CREATE TABLE  
CREATE FUNCTION  
CREATE TRIGGER  
total time: 10 ms
```

删除sc444表后在bk444中保存信息

```
mydb=# select count(*) from bk444;
count
-----
100
(1 row)
```

```
mydb=# select * from bk444;
 s#      | c#      | grade |      ddate
-----+-----+-----+-----
13776301 | FL-94   |       | 2025-06-07 00:29:45
30487217 | MA-26   |       | 2025-06-07 00:29:46
28288997 | CE-11   |       | 2025-06-07 00:29:48
40762427 | AT-50   |       | 2025-06-07 00:29:50
42999349 | HS-10   |       | 2025-06-07 00:29:52
26998389 | OE-18   |       | 2025-06-07 00:29:53
41667882 | HS-13   |       | 2025-06-07 00:29:55
28050863 | OE-14   |       | 2025-06-07 00:29:56
43878856 | PH-11   |       | 2025-06-07 00:29:58
36239213 | EM-07   |       | 2025-06-07 00:29:59
19297088 | EE-10   |       | 2025-06-07 00:30:01
```

## 七、关于实验的意见与建议

作为网络安全专业的学生，可以感知到课业压力明显高于计算机专业的同学（毕业学分要求多20）。本实验内容偏多，占用较多时间，且与日后的工作与学习关系不大。目前主流的关系型数据库是MySQL（MariaDB）、PostgreSQL、Oracle Database等，网安专业学生在进行网络渗透测试时也主要面对这些数据库，因此更应该多学习和熟悉这些主流数据库

建议适度降低对网安专业学生的要求，不强制要求使用openGauss，而允许使用MySQL等更主流的数据库，同时保留本地部署、触发器等高级要求